



UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Geração de código em tempo de execução

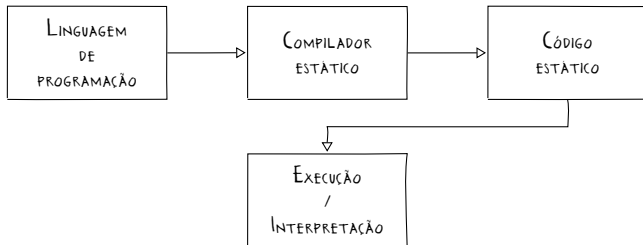
Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

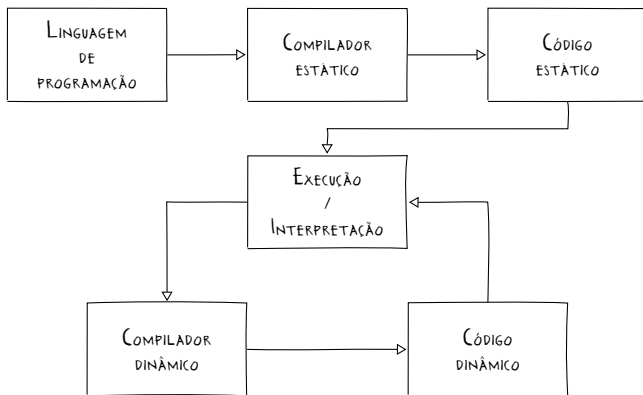
Introdução

- ▶ Por que gerar código em tempo de execução?
 - ▶ Permite a otimização adaptativa de código de nativo gerado a partir de informações da execução



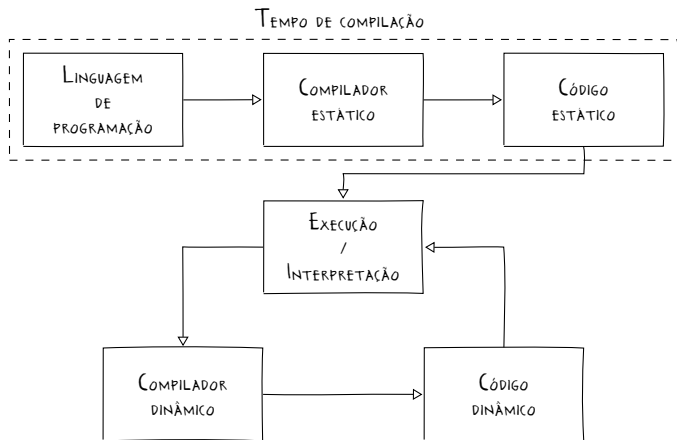
Introdução

- ▶ Por que gerar código em tempo de execução?
 - ▶ Permite a otimização adaptativa de código de nativo gerado a partir de informações da execução



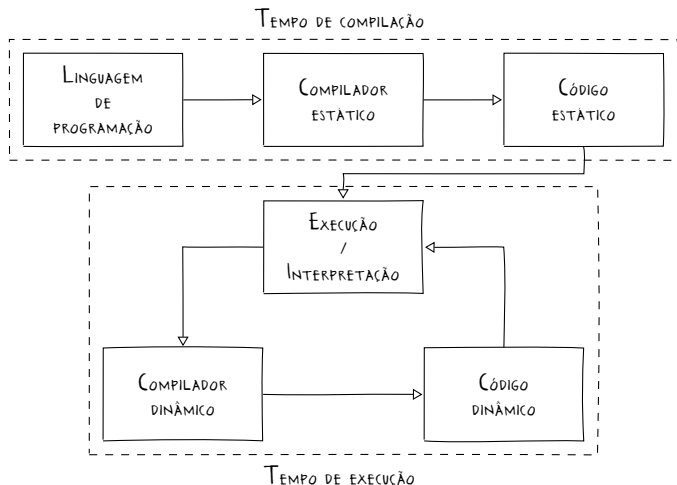
Introdução

- ▶ Por que gerar código em tempo de execução?
 - ▶ Permite a otimização adaptativa de código de nativo gerado a partir de informações da execução



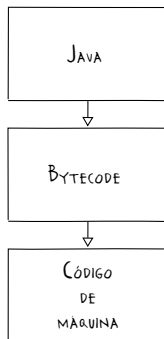
Introdução

- ▶ Por que gerar código em tempo de execução?
 - ▶ Permite a otimização adaptativa de código de nativo gerado a partir de informações da execução



Introdução

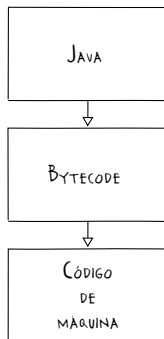
- ▶ *Ahead-Of-Time* (AOT)
 - ▶ A linguagem de programação é estaticamente traduzida para o código de máquina da arquitetura alvo



Introdução

- ▶ *Ahead-Of-Time* (AOT)

- ▶ A linguagem de programação é estaticamente traduzida para o código de máquina da arquitetura alvo

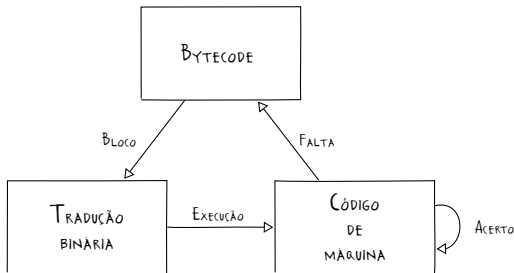


- ▶ São realizadas otimizações por inferência para a arquitetura alvo em tempo de compilação, para reduzir o custo de execução da aplicação

Introdução

► *Just-In-Time* (JIT)

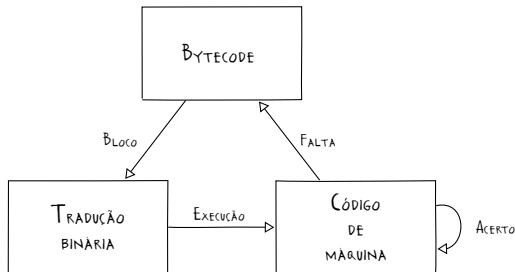
- É feita a interpretação do código intermediário ou de máquina que é dinamicamente traduzido sob demanda em código de máquina do hospedeiro



Introdução

► *Just-In-Time* (JIT)

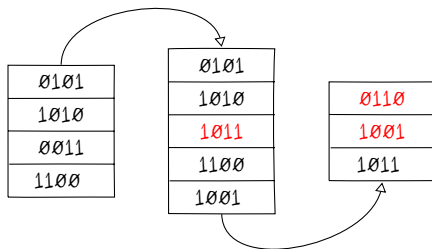
- É feita a interpretação do código intermediário ou de máquina que é dinamicamente traduzido sob demanda em código de máquina do hospedeiro



- As otimizações adaptativas e a tradução binária ocorrem em tempo de execução, com expectativa de oferecer uma melhoria de desempenho superior ao custo adicional gerado na execução

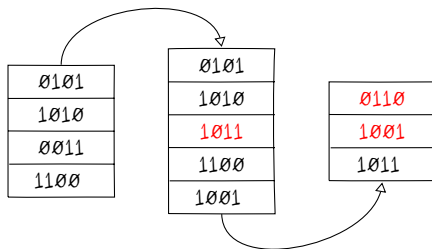
Introdução

- ▶ Código automodificável
 - ▶ Possibilita que as instruções do programa sejam modificadas durante a execução



Introdução

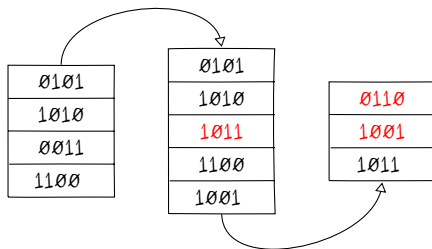
- ▶ Código automodificável
 - ▶ Possibilita que as instruções do programa sejam modificadas durante a execução



- ▶ Aplicações
 - ▶ Geração de código em tempo de execução

Introdução

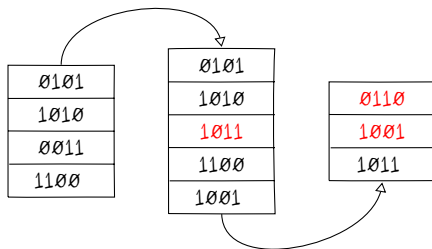
- ▶ Código automodificável
 - ▶ Possibilita que as instruções do programa sejam modificadas durante a execução



- ▶ Aplicações
 - ▶ Geração de código em tempo de execução
 - ▶ Computação evolucionária

Introdução

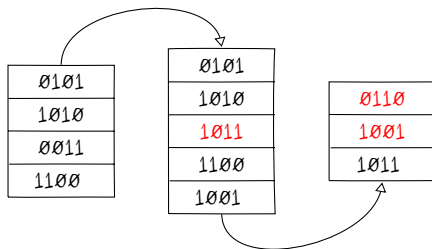
- ▶ Código automodificável
 - ▶ Possibilita que as instruções do programa sejam modificadas durante a execução



- ▶ Aplicações
 - ▶ Geração de código em tempo de execução
 - ▶ Computação evolucionária
 - ▶ Prevenção de engenharia reversa

Introdução

- ▶ Código automodificável
 - ▶ Possibilita que as instruções do programa sejam modificadas durante a execução



- ▶ Aplicações
 - ▶ Geração de código em tempo de execução
 - ▶ Computação evolucionária
 - ▶ Prevenção de engenharia reversa
 - ▶ ...

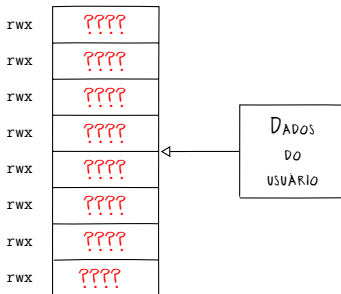
Introdução

- ▶ Questões de segurança
 - ▶ A execução de código gerado pelo usuário em tempo de execução, sem as devidas medidas de proteção, pode representar uma grave ameaça para a integridade e segurança do sistema

rwX	0101
rwX	1010
rwX	0011
rwX	1100
rwX	0110
rwX	1001
rwX	1101
rwX	1011

Introdução

- ▶ Questões de segurança
 - ▶ A execução de código gerado pelo usuário em tempo de execução, sem as devidas medidas de proteção, pode representar uma grave ameaça para a integridade e segurança do sistema



O que pode dar 3rr4d0?

Introdução

- ▶ Bibliotecas e ferramentas para geração de código
 - ▶ GNU lightning
 - ▶ libJIT
 - ▶ LLVM
 - ▶ ...

Interpretação

► Exponenciação modular $a = b^n \bmod p$

```
1 // Entrada e saída padrão
2 #include <stdio.h>
3 // Padronização de tipos inteiros
4 #include <stdint.h>
5 // Função de exponenciação modular
6 const uint64_t expmod(const uint64_t b, const uint64_t
    n, const uint64_t p) {
7     // Resultado de 128 bits
8     __uint128_t a = 1;
9     // a = (b ^ n) mod p
10    for(uint64_t i = 0; i < n; i++)
11        a = (a * b) % p;
12    // Retornando resultado de 64 bits
13    return a;
14 }
... ..
```

Interpretação

► Exponenciação modular $a = b^n \bmod p$

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
15 // Função principal
16 int main() {
17     // Imprimindo resultado
18     printf("2^123456789 mod 18446744073709551533 = \n", expmod(2, 123456789,
18         18446744073709551533ULL));
19     // Retornando sucesso
20     return 0;
21 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
```

Interpretação

► Exponenciação modular $a = b^n \bmod p$

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
15 // Função principal
16 int main() {
17     // Imprimindo resultado
18     printf("2^123456789 mod 18446744073709551533 =\n",
19           expmod(2, 123456789,
20                18446744073709551533ULL));
19     // Retornando sucesso
20     return 0;
21 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
2^123456789 mod 18446744073709551533 = 7398179800986853190
```

Interpretação

► Exponenciação modular $a = b^n \bmod p$

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
15 // Função principal
16 int main() {
17     // Imprimindo resultado
18     printf("2^123456789 mod 18446744073709551533 =\n",
           expmod(2, 123456789,
           18446744073709551533ULL));
19     // Retornando sucesso
20     return 0;
21 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
2^123456789 mod 18446744073709551533 = 7398179800986853190
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 1.868 s +- 0.001 s [User: 1.868 s, System: 0.000 s]
Range (min ... max): 1.867 s ... 1.869 s 10 runs
```

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
 - ▶ Linguagem de domínio específico (SDL)

```
1 // Alocação de variáveis
2 set64 b 2
3 set64 n 123456789
4 set64 p 18446744073709551533
5 set128 a 1
6 // a = (a ^ n) mod p
7 from set64 i 0 to n
8     mul128 a a b
9     mod128 a a p
10 // Retorno de resultado
11 ret64 a
```

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
 - ▶ Linguagem de domínio específico (SDL)

```
1 // Alocação de variáveis
2 set64 b 2
3 set64 n 123456789
4 set64 p 18446744073709551533
5 set128 a 1
6 // a = (a ^ n) mod p
7 from set64 i 0 to n
8     mul128 a a b
9     mod128 a a p
10 // Retorno de resultado
11 ret64 a
```

Conversão da descrição para
uma sequência de bytes (*bytecode*)

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
 - ▶ Interpretação das operações do *bytecode*

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
5 // Função de interpretação
6 const uint64_t interpret(uint8_t* code, uint32_t n) {
...
15     // Retornando valor da execução
16     return value;
17 }
...
...
```


Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
- ▶ Interpretação das operações

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
131 // Função principal
132 int main() {
133     // Arquivo de programa
134     FILE* file = fopen("program", "r");
135     uint8_t code[128] = { 0 };
136     uint32_t byte = 0, n = 0;
137     // Leitura do bytecode
138     while(fscanf(file, "%X", &byte) == 1)
139         code[n++] = byte;
140     // Interpretação do código
141     printf("2^123456789 mod 18446744073709551533 = \n",
           %lu\n", interpret(code, n));
142     // Retornando sucesso
143     return 0;
144 }
```

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
 - ▶ Interpretação das operações

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
...
132 int main() {
...
...
140 // Interpretação do código
141 printf("2^123456789 mod 18446744073709551533 =
    %lu\n", interpret(code, n));
142 // Retornando sucesso
143 return 0;
144 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
```

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
 - ▶ Interpretação das operações

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
...
132 int main() {
...
...
140 // Interpretação do código
141 printf("2^123456789 mod 18446744073709551533 =
    %lu\n", interpret(code, n));
142 // Retornando sucesso
143 return 0;
144 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
2^123456789 mod 18446744073709551533 = 7398179800986853190
```

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
 - ▶ Interpretação das operações

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
...
132 int main() {
...
...
140 // Interpretação do código
141 printf("2^123456789 mod 18446744073709551533 =
    %lu\n", interpret(code, n));
142 // Retornando sucesso
143 return 0;
144 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
2^123456789 mod 18446744073709551533 = 7398179800986853190
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 2.796 s +- 0.082 s [User: 2.795 s, System: 0.000 s]
Range (min ... max): 2.747 s ... 2.984 s 10 runs
```

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
- ▶ Implementação fixa (*hardcoded*)

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
9 const uint64_t hardcoded() {
10     // Alocação de variáveis
11     v64['b'] = 2;
12     v64['n'] = 123456789;
13     v64['p'] = 18446744073709551533ULL;
14     v128['a'] = 1;
15     // a = (a ^ n) mod p
16     for(v64['i'] = 0; v64['i'] < v64['n']; v64['i']++) {
17         v128['a'] = v128['a'] * v64['b'];
18         v128['a'] = v128['a'] % v64['p'];
19     }
20     // Retorno de resultado
21     return v128['a'];
22 }
...
```

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
 - ▶ Implementação fixa (*hardcoded*)

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
23 // Função principal
24 int main() {
25     // Execução do código
26     printf("2^123456789 mod 18446744073709551533 =
        %llu\n", hardcoded());
27     // Retornando sucesso
28     return 0;
29 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
```

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
 - ▶ Implementação fixa (*hardcoded*)

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
23 // Função principal
24 int main() {
25     // Execução do código
26     printf("2^123456789 mod 18446744073709551533 =
        %llu\n", hardcoded());
27     // Retornando sucesso
28     return 0;
29 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
2^123456789 mod 18446744073709551533 = 7398179800986853190
```

Interpretação

- ▶ Exponenciação modular $a = b^n \bmod p$
 - ▶ Implementação fixa (*hardcoded*)

```
1 // Entrada e saída padrão
2 #include <stdio.h>
...
23 // Função principal
24 int main() {
25     // Execução do código
26     printf("2^123456789 mod 18446744073709551533 =
        %llu\n", hardcoded());
27     // Retornando sucesso
28     return 0;
29 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
2^123456789 mod 18446744073709551533 = 7398179800986853190
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 1.847 s +- 0.001 s [User: 1.845 s, System: 0.002 s]
Range (min ... max): 1.846 s ... 1.848 s 10 runs
```


Exercício

- ▶ Implemente um interpretador para a linguagem de domínio específico utilizada para implementação da exponenciação modular
 - ▶ Calcule o tempo de execução de seu interpretador e compare com as outras implementações
 - ▶ Explique porque a interpretação, apesar da flexibilidade, impacta no desempenho de execução

Geração de código

► AOT × Interpretação

- A utilização de técnicas de JIT permite combinar o desempenho da execução nativa com a portabilidade de uma implementação que independe da arquitetura alvo



Geração de código

► AOT × Interpretação

- A utilização de técnicas de JIT permite combinar o desempenho da execução nativa com a portabilidade de uma implementação que independe da arquitetura alvo



- A tradução é feita sob demanda, considerando o fluxo de execução do programa

Geração de código

► AOT × Interpretação

- A utilização de técnicas de JIT permite combinar o desempenho da execução nativa com a portabilidade de uma implementação que independe da arquitetura alvo



- A tradução é feita sob demanda, considerando o fluxo de execução do programa
- Quanto mais vezes um trecho do código for referenciado (*hot spot*), maior é o desempenho do JIT, podendo superar a compilação estática

Geração de código

► Alocação de memória executável

```
1  /**
2   *  A função mmap cria um novo mapeamento da
      memória no processo
3   *  @param addr    Endereço inicial, se for NULL, é
      gerado pelo SO
4   *  @param length  Tamanho do mapeamento de memória
5   *  @param prot    Proteção de acesso aos dados
      (PROT_EXEC, PROT_NONE, PROT_READ e PROT_WRITE)
6   *  @param flags   Tipo de mapeamento
      (MAP_ANONYMOUS, MAP_PRIVATE e MAP_SHARED)
7   *  @param fd      Descritor de arquivo
8   *  @param offset  Valor múltiplo do tamanho da
      página
9   *  @return        Retorna o endereço de memória
      (sucesso) ou (void*)(-1) (erro)
10  */
11  void* mmap(void* addr, size_t length, int prot, int
      flags, int fd, off_t offset);
```

Geração de código

► Alocação de memória executável

```
1  /**
2   *   As funções mprotect e munmap modificam as
3   *   proteções de acesso e desalocam o mapeamento de
4   *   memória no processo
5   *   @param addr    Endereço inicial
6   *   @param length  Tamanho do mapeamento de memória
7   *   @param prot     Proteção de acesso aos dados
8   *                   (PROT_EXEC, PROT_NONE, PROT_READ e PROT_WRITE)
9   *   @return        Retorna sucesso (0) ou erro (-1)
10  */
11  int mprotect(void* addr, size_t len, int prot);
12  int munmap(void* addr, size_t length);
```

Geração de código

► Alocação de memória executável

```
1 // Gerenciamento de memória
2 #include <sys/mman.h>
...
8 // rdi = b, rsi = n, rdx = p
9 uint8_t code[] = {
10     0x55,                // push rbp
11     0x48, 0x89, 0xE5,     // mov rbp, rsp
12     0x48, 0x89, 0xD1,     // mov rcx, rdx (p)
13     0x66, 0xB8, 0x01, 0x00, // mov ax, 1      (a = 1)
14     0x4D, 0x31, 0xC0,     // xor r8, r8     (i = 0)
15     ...
23     0x5D,                // pop rbp
24     0xC3                 // ret
25 };
...
...
```

Geração de código

► Alocação de memória executável

```
1 // Gerenciamento de memória
2 #include <sys/mman.h>
3 ...
4
5 // rdi = b, rsi = n, rdx = p
6 uint8_t code[] = {
7     ...
8     0x49, 0x39, 0xF0, // cmp r8, rsi (i ? n)
9     0x0F, 0x83, 0x11,
10    0x00, 0x00, 0x00, // jae 0x11      (i >= n end)
11    0x48, 0xF7, 0xE7, // mul rdi      (a = a * b)
12    0x48, 0xF7, 0xF1, // div rcx      (rdx = a % p)
13    0x48, 0x89, 0xD0, // mov rax, rdx (rax = rdx)
14    0x49, 0xFF, 0xC0, // inc r8      (i++)
15    0xE9, 0xE6, 0xFF,
16    0xFF, 0xFF,      // jmp -0x1A    (loop again)
17    ...
18 };
19 ...
```


Geração de código

► Alocação de memória executável

```
1 // Gerenciamento de memória
2 #include <sys/mman.h>
...
18 // Função principal
19 int main() {
20     // Tamanho da página de memória para alocação
21     uint32_t length = sysconf(_SC_PAGE_SIZE);
22     void* memory = mmap(0, length, PROT_NONE,
23         MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
24     // Escrita -> Cópia -> Executável
25     mprotect(memory, length, PROT_WRITE);
26     memcpy(memory, (void*)(code), sizeof(code));
27     mprotect(memory, length, PROT_EXEC);
28     // Ponteiro de função
29     const uint64_t(*jit)(const uint64_t, const
30         uint64_t, const uint64_t) = (const
31         uint64_t*)(const uint64_t, const uint64_t,
32         const uint64_t))(memory);
33     ...
39 }
```

Geração de código

► Execução de código gerado em memória

```
1 // Gerenciamento de memória
2 #include <sys/mman.h>
...
18 int main() {
...
    ...
29 // Execução do código e liberação de memória
30 printf("2^123456789 mod 18446744073709551533 =
    %lu\n", (*jit)(2, 123456789,
    18446744073709551533ULL));
31 munmap(memory, length);
...
    ...
39 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
```

Geração de código

► Execução de código gerado em memória

```
1 // Gerenciamento de memória
2 #include <sys/mman.h>
...
18 int main() {
...
    ...
29 // Execução do código e liberação de memória
30 printf("2^123456789 mod 18446744073709551533 = %lu\n",
        (*jit)(2, 123456789,
        18446744073709551533ULL));
31 munmap(memory, length);
...
    ...
39 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
2^123456789 mod 18446744073709551533 = 7398179800986853190
```

Geração de código

► Execução de código gerado em memória

```
1 // Gerenciamento de memória
2 #include <sys/mman.h>
...
18 int main() {
...
    ...
29 // Execução do código e liberação de memória
30 printf("2^123456789 mod 18446744073709551533 = %lu\n",
        (*jit)(2, 123456789,
        18446744073709551533ULL));
31 munmap(memory, length);
...
    ...
39 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ ./exemplo.elf
2^123456789 mod 18446744073709551533 = 7398179800986853190
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 1.837 s +- 0.001 s [User: 1.835 s, System: 0.001 s]
Range (min ... max): 1.836 s ... 1.838 s 10 runs
```

Geração de código

► Análise comparativa de desempenho

Abordagem	Tempo médio (s)	Desempenho
AOT	1,868	1,00x
<i>Hardcoded</i>	1,847	1,01x
Interpretação	2,796	0,67x
JIT	1,837	1,02x

Exercício

- ▶ Implemente a interpretação e a tradução em tempo de execução do código de máquina para arquitetura *little-endian* PicoQuickProcessor de 32 bits
 - ▶ Com 16 registradores de propósito geral e uma memória Von Neumann com 256 bytes
 - ▶ A execução do programa finaliza quando um endereço de instrução é acessado além da memória
 - ▶ Revise as convenções da ABI e compare o desempenho do interpretador versus do código gerado

Exercício

► Especificação da arquitetura PicoQuickProcessor

mov r_x , i16	0x00	$r_{x3-0} : -$	i_{15-0}		$r_x = i_{15}^{16} : i_{15-0}$
mov r_x , r_y	0x01	$r_{x3-0} : r_{y3-0}$	-	-	$r_x = r_y$
mov r_x , [r_y]	0x02	$r_{x3-0} : r_{y3-0}$	-	-	$r_x = \text{MEM}[r_y]$
mov [r_x], r_y	0x03	$r_{x3-0} : r_{y3-0}$	-	-	$\text{MEM}[r_x] = r_y$
cmp r_x , r_y	0x04	$r_{x3-0} : r_{y3-0}$	-	-	$r_x \lt \rightarrow r_y$
jmp i16	0x05	-	i_{15-0}		$\text{pc} += 4 + i_{15}^{16} : i_{15-0}$
jg i16	0x06	-	i_{15-0}		$g \rightarrow \text{pc} += 4 + i_{15}^{16} : i_{15-0}$
jl i16	0x07	-	i_{15-0}		$l \rightarrow \text{pc} += 4 + i_{15}^{16} : i_{15-0}$
je i16	0x08	-	i_{15-0}		$e \rightarrow \text{pc} += 4 + i_{15}^{16} : i_{15-0}$
add r_x , r_y	0x09	$r_{x3-0} : r_{y3-0}$	-	-	$r_x += r_y$
sub r_x , r_y	0x0A	$r_{x3-0} : r_{y3-0}$	-	-	$r_x -= r_y$
and r_x , r_y	0x0B	$r_{x3-0} : r_{y3-0}$	-	-	$r_x \&= r_y$
or r_x , r_y	0x0C	$r_{x3-0} : r_{y3-0}$	-	-	$r_x = r_y$
xor r_x , r_y	0x0D	$r_{x3-0} : r_{y3-0}$	-	-	$r_x \wedge= r_y$
sal r_x , i5	0x0E	$r_{x3-0} : -$	-	i_{4-0}	$r_x \ll= i_{4-0}$
sar r_x , i5	0x0F	$r_{x3-0} : -$	-	i_{4-0}	$r_x \gg= i_{4-0}$